16

Использование поддержки глобализации

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Рассматриваемые вопросы

- Определение подходящей кодировки базы данных, удовлетворяющей вашим бизнес-требованиям
- Общие сведения о конфигурации поддержки глобализации
- Настройка языковых особенностей на уровне базы данных и для отдельных сеансов
- Задание в запросах различных лингвистических сортировок
- Выбор данных, соответствующих строке поиска, в которой игнорируется регистр символов и ударение

ORACLE

16-2 Copyright © 2006, Oracle. All rights reserved.

Рассматриваемые вопросы

Дополнительные сведения см. в документе *Oracle Database Globalization Support Guide*.

Что необходимо знать каждому АБД

- Что такое кодировка?
- Как используется кодировка?
- Проблемы, которые следует избегать
- Как правильно выбрать кодировку
- Получение сведений о кодировках
- Задание особенностей функционирования, зависящих от языка
- Использование лингвистического поиска и сортировки
- Использование преобразований данных

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Что такое кодировка?

Oracle поддерживает разные классы схем кодирования символов:

- **однобайтные кодировки**
  - **7-битовая**
  - **8-битовая**
- **многобайтовые кодировки, включая Unicode**

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Что такое кодировка?

При обработке символов в компьютерной системе используются числовые коды символов, а не их графическое представление. *Схема кодирования символов* (*encoded character set*) задает соответствие между символами, которые могут приниматься и выводиться компьютером или терминалом, и их кодовым представлением. База данных Oracle в настоящее время поддерживает около 30 *с*хем кодирования символов и при этом значительно большее количество языков и территорий (около 100). Это возможно, потому что Unicode – универсальная кодировка, содержащая большинство основных *символов, используемых при письме* (*scripts*) в современном мире.

Различные кодировки поддерживают различные наборы символов (например кириллицу, латиницу). Поскольку кодировки обычно основываются на определенной *письменной системе* (*script*), они могут поддерживать несколько языков. Однако такие кодировки ограничены в том смысле, что они соответствуют группам языков, основанным на схожих наборах символов. Универсальные же кодировки используют большинство письменных систем, применяемых в современных языках и дают решение для одновременной поддержки нескольких языков. Дополнительные сведения о стандартах Unicode см. на Web-сайте http:www.unicode.org.

База данных Oracle поддерживает различные схемы кодирования символов:

- однобайтовую;
- многобайтную переменной ширины;
- всемирную.

Что такое кодировка? (продолжение)

Однобайтовые кодировки

В однобайтовых кодировках каждый символ занимает один байт. Однобайтовые 7-битовые кодировки могут определять до 128 (2^7) символов; однобайтовые 8-битовые кодировки могут определять до 256 (2^8) символов.

Примеры однобайтовых кодировок

7-битовая кодировка:

- American Standard Code for Information Interchange (Американский стандартный код обмена информацией – ASCII) 7-bit American (`US7ASCII`)
- ASCII 7-bit Yugoslavian (`YUG7ASCII`)
- DEC VT100 7-bit French (`F7DEC`)

8-битовая кодировка:

- ISO (International Organization for Standards – Международная организация по стандартизации) 8859-1 West European (WE8ISO8859P1)
- DEC 8-bit West European (`WE8DEC`)
- Extended Binary Coded Decimal Interchange Code (расширенный двоично-десятичный код обмена информацией – EBCDIC) Code Page 1144 8-bit Italian (`I8EBCDIC1144`)

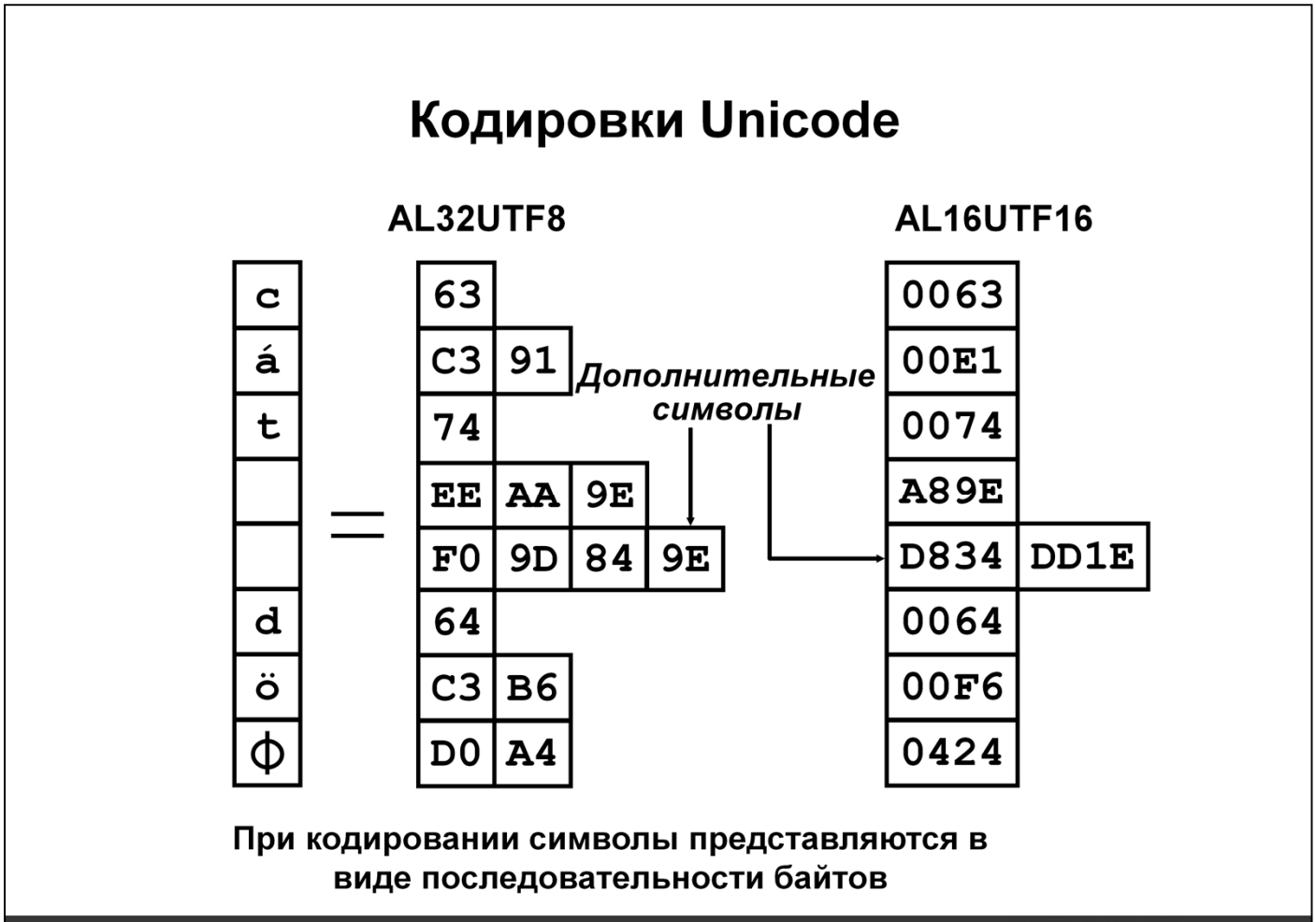
  Примечание. ASCII-кодировки поддерживаются только на платформах, основанных на использовании ASCII. Кодировки EBCDIC используются только на платформах, основанных на использовании EBCDIC.

Многобайтовые кодировки

В многобайтовых кодировках на один символ отводится один или несколько байтов. Обычно многобайтовые кодировки используются для поддержки азиатских языков. В некоторых многобайтовых кодировках значение старшего бита используется для указания, является ли байт одиночным или входит в набор байтов, представляющих символ. Другие же кодировки разделяют однобайтовые и многобайтовые символы. Устройство посылает управляющий код, показывающий, что следующие пары байтов будут интерпретироваться как представление одного символа до тех пор, пока не поступит управляющий код возврата к стандартной кодировке. Кодировки, использующие управляющий код, в основном применяются на платформах IBM.

Примеры многобайтовых кодировок переменной ширины

- Shift-JIS 16-bit Japanese (`JA16SJIS`)
- MS Windows Code Page 950 with Hong Kong Supplementary Character Set HKSCS-2001 (`ZHT16HKSCS`)
- Unicode 4.0 UTF-8 Universal character set (`AL32UTF8`)



Кодировки Unicode

Unicode – это всемирный стандарт кодирования символов, который позволяет хранить информацию на различных языках в одной схеме кодирования. Unicode предоставляет уникальный код для каждого символа независимо от платформы, программы или языка. Стандарт Unicode был одобрен многими производителями программного и аппаратного обеспечения. В настоящее время Unicode поддерживается многими операционными системами и браузерами. Стандарты XML, Java, JavaScript, LDAP и WML требуют использования Unicode. Кроме того, стандарт Unicode согласован с стандартом ISO/IEC 10646.

Кодировка AL32UTF8

AL32UTF8 – кодировка Unicode, использующая 8-битовые кодовые последовательности. Она относится к типу кодировок переменной длины и представляет собой строгое надмножество над ASCII. Это означает, что каждому символу в 7-bit ASCII соответствует такое же кодовое значение в AL32UTF8.

Один символ в этой кодировке Unicode может быть предоставлен 1, 2, 3 или 4 байтами. Символы европейских национальных алфавитов поддерживаются с использованием 1 или 2 байтов; Символы азиатских национальных алфавитов – 3 байтами, а дополнительные символы – 4 байтами.

Кодировки Unicode (продолжение)

Кодировка AL16UTF16

AL16UTF16 – кодировка Unicode, использующая 16-битовые кодовые последовательности.

В этой системе кодирования один символ может быть представлен 2 или 4 байтами. Символы европейских алфавитов (а также ASCII) и большинства азиатских алфавитов представлены 2 байтами. Дополнительные символы отображаются 4 байтами. AL16UTF16 – основная кодировка Unicode для Microsoft Windows 2000 и Windows XP.

Дополнительные символы

В первоначальной версии Unicode использовался 2-байтовый формат кодирования. Такое использование 16 бит для каждого кодируемого элемента позволяет представить до 65536 символов. Однако требуется поддерживать значительно большое количество символов. Например, только сообщество говорящих на китайском использует более 55000 символов.

В таких языках, как китайский, японский и корейский еще не закодированы десятки тысяч идеограмм. И несмотря на то, что многие из этих символов используются редко, они все еще представлены в документах, которые должны сохраняться в электронном виде.

Для удовлетворения этого требования в стандарте Unicode определяются *дополнительные символы* (*supplementary characters*). Применяя два 16-битовых кодовых указателя (их называют также *заменяющими парами* (*surrogate pairs*)) для представления одного символа, можно дополнительно определить до 1 048 576 символов.

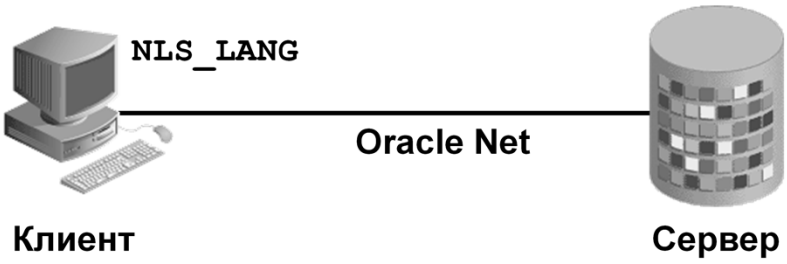
Первая группа дополнительных символов (4944 символа) была добавлена в стандарт Unicode 3.1, выпущенный в марте 2001 года. Вместе с уже существовавшими в Unicode 3.0 49194 символами общее число символов, закодированных в Unicode 3.1, составляет сейчас 94140. Это вносит большую сложность в стандарт Unicode. Однако это значительно проще, чем сопровождать большое количество отдельных кодировок. База данных Oracle 10*g* поддерживает стандарт Unicode 4.0.

Примечание: кодировки UTF-16 и UTF-8 (с дефисом) относятся к кодировкам стандарта Unicode; `UTF8`, `AL32UTF8` и `AL16UTF16` (без дефиса) относятся к кодировкам Oracle, основанным на стандарте Unicode.

Примечание: дополнительные сведения о поддержке Oracle стандарта Unicode см. в документе *Oracle Database Globalization Support Guide 10g Release 2 (10.2)*.

Как используются кодировки?

- **Oracle Net сравнивает установленное на клиенте значение переменной среды `NLS_LANG` с кодировкой на сервере.**
- **При необходимости производится автоматическое и прозрачное преобразование.**



Copyright © 2006, Oracle. All rights reserved.

Как используются кодировки?

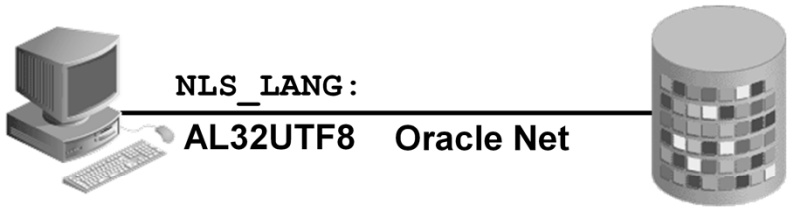
`NLS_LANG` задает схему кодирования символов для терминала клиента. Разные клиенты могут использовать различные кодировки. Если кодировки клиента и сервера отличаются, то при передаче данных между ними происходит автоматическая перекодировка. Кодировка базы данных должна быть надмножеством, или эквивалентом, всех клиентских кодировок. Перекодировка выполняется в прозрачном для клиентского приложения режиме.

Когда кодировки базы данных и клиента совпадают, Oracle считает, что информация принимается и передается в той же схеме кодирования и не выполняет никаких проверок и преобразований.

Преобразование из одной кодировки в другую может потребоваться в среде клиент-сервер, когда клиентское приложение размещается на платформе, отличной от серверной, и не использует такую же схему кодирования. Символьные данные, передаваемые между клиентом и сервером, должны быть преобразованы из одной схемы кодирования в другую. Символьное преобразование происходит автоматически и прозрачно с помощью Oracle Net.

Такие проблемы следует избегать

Пример:



Клиент
Русская версия Windows
Кодовая страница: CL8MSWIN1251

Сервер
Кодировка базы данных:
AL32UTF8

Никакие преобразования не производятся, потому что складывается впечатление, что они не нужны.

Проблема: недостоверные данные вносятся в БД.

ORACLE

16-9 Copyright © 2006, Oracle. All rights reserved.

Такие проблемы следует избегать

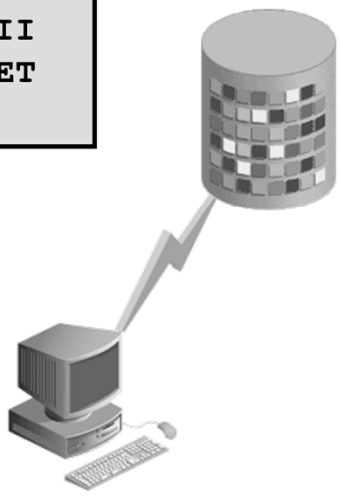
Недостоверные данные вносятся в базу данных, когда на клиенте неверно установлена переменная среды NLS_LANG. Значение NLS_LANG должно отражать схему кодирования поступающих на сервер данных.

- Если переменная среды NLS_LANG установлена правильно, тогда база данных может автоматически преобразовывать данные, поступающие из клиентской операционной системы.
- Если переменная среды NLS_LANG установлена неправильно, тогда база данных неверно преобразует поступающие данные.

Например, предположим кодировка базы данных AL32UTF8, на клиенте установлена русская версия операционной системы Windows (кодовая страница: CL8MSWIN1251) и на клиенте же в переменной среды NLS_LANG указана кодировка AL32UTF8. Данные поступают в БД в кодировке CLE8MSWIN1251 и не преобразуются в AL32UTF8, поскольку заданная в NLS_LANG установка соответствует кодировке базы данных. Поэтому база данных Oracle полагает, что преобразования не нужны и неверные данные вносятся в базу данных.

Пример еще одной проблемы

```
CREATE DATABASE ...
CHARACTER SET US7ASCII
NATIONAL CHARACTER SET
UTF8 ...
```



```
% export NLS_LANG= SIMPLIFIED
CHINESE_HONGKONG.US7ASCII
```

ORACLE

16-10 Copyright © 2006, Oracle. All rights reserved.

Пример еще одной проблемы

Пусть кодировка базы данных `US7ASCII` и на терминале клиента используется Simplified Chinese Windows, т.е. значение параметра `NLS_LANG` в окружении клиента `SIMPLIFIED CHINESE_HONGKONG.US7ASCII`. В этом случае возможны ситуации, когда пользователь сохранит многобайтные символы Simplified Chinese в одном байте базы данных. Oracle будет обрабатывать символы как однобайтные символы кодировки `US7ASCII`, поэтому все SQL-функции манипулирования строками (например, `SUBSTR`, `LENGTH)` будут выполнять обработку на основе байта, а не символа. Все не ASCII символы могут быть потеряны после экспорта и импорта в другую базу данных.

Выбор кодировки

- Рассмотрение альтернативных вариантов
- Выбор кодировки, удовлетворяющей вашим бизнес-требованиям сегодня и в будущем
- Задание кодировки
- Изменение кодировки после создания базы данных

ORACLE

16-11 Copyright © 2006, Oracle. All rights reserved.

Выбор кодировки

Для достижения наилучшей производительности выбирайте кодировку, которая устраняет преобразование символов различных кодировок и использует наиболее эффективную схему кодирования для требуемого языка. Однобайтовые кодировки наиболее оптимальны с точки зрения производительности и требуемого пространства по сравнению с многобайтовыми. Однако однобайтовые кодировки охватывают ограниченный набор используемых языков.

Для правильного выбора кодировки базы данных оцените ваши текущие и будущие бизнес-требования, а также технические требования (например, в соответствие с стандартами XML и Java требуется использовать Unicode). В общем случае Oracle рекомендует использовать Unicode для всех новых баз данных, поскольку это наиболее гибкая кодировка, которая также позволит избежать преобразований в будущем.

Для указания кодировки используется команда `CREATE DATABASE`. В ней в предложении `CHARACTER SET` объявляется *кодировка базы данных* и в предложении `NATIONAL CHARACTER SET` – *национальная кодировка.* Если `NATIONAL CHARACTER SET` не указывается, тогда по умолчанию задается национальная кодировка `AL16UTF16`.

После создания базы данных может потребоваться изменить кодировку БД. Это может быть вызвано появлением непредусмотренных заранее требований, например, необходимостью поддержки новых источников данных (XA, хранилище данных и т.д.). Часто такое изменение может привести к значительным временным затратам и оказаться дорогостоящим процессом. В большинстве случаев понадобится выполнить полный экспорт/импорт, чтобы соответствующим образом преобразовать данные из старой кодировки в новую.

Кодировки базы данных и национальные кодировки

Кодировки базы данных	Национальные кодировки
Задаются во время создания	Задаются во время создания
Не могут быть изменены без повторного создания с некоторыми исключениями	Могут быть изменены
Хранят столбцы данных типа `CHAR`, `VARCHAR2`, `CLOB`, `LONG`	Хранят столбцы данных типа `NCHAR`, `NVARCHAR2`, `NCLOB`
Могут хранить набор символов переменной ширины	Могут хранить набор символов Unicode, используя `AL16UTF16` или `UTF8`

ORACLE

16-12 Copyright © 2006, Oracle. All rights reserved.

Кодировки базы данных и национальные кодировки

Поскольку кодировка базы данных используется для идентификации и хранения исходного кода SQL и PL/SQL, она должна в зависимости от платформы включать в себя 7-битную кодировку ASCII или EBCDIC. Поэтому многобайтная кодировка фиксированной ширины не может использоваться в качестве кодировки базы данных, а может использоваться только в качестве национальной кодировки.

Национальная кодировка – это альтернативная кодировка, позволяющая хранить символьные данные в кодировке Unicode, если в качестве кодировки базы данных не определена Unicode. Для хранения типов данных SQL `NCHAR`, `NVARCHAR2` и `NCLOB` используется только кодировка Unicode. Можно выбрать одну из двух таких кодировок: `UTF8` или `AL16UTF16`.

Получение информации о кодировках

```
SQL> SELECT parameter, value
 2   FROM nls_database_parameters
 3   WHERE parameter LIKE '%CHARACTERSET%';

PARAMETER                    VALUE
-----------------------    -------------
NLS_CHARACTERSET             CL8MSWIN1251
NLS_NCHAR_CHARACTERSET AL16UTF16

2 rows selected.
```

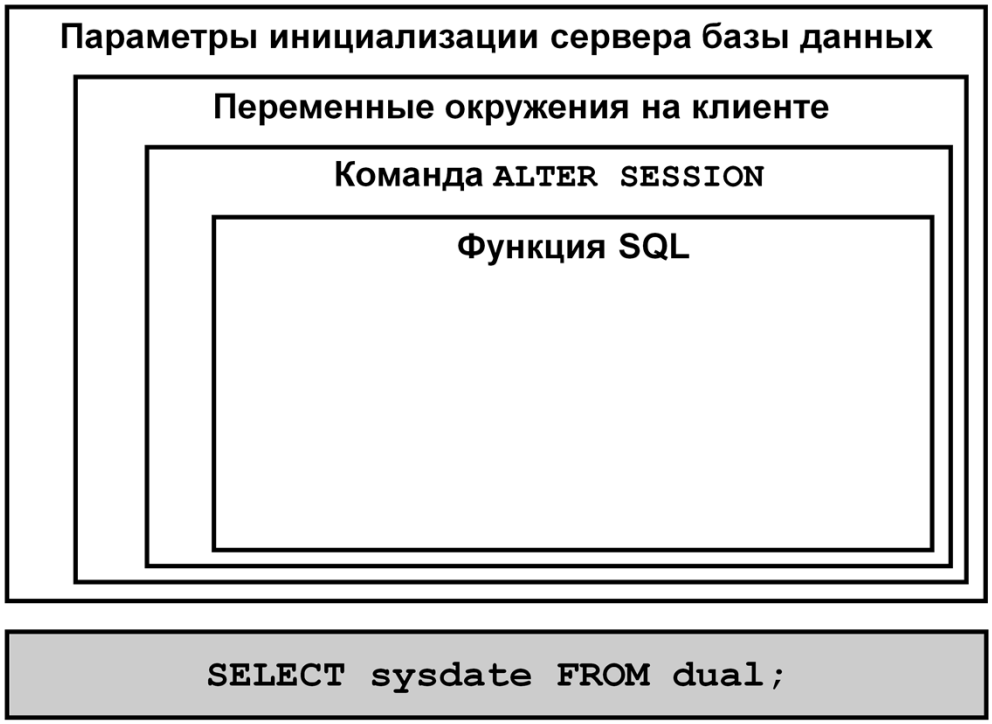
ORACLE

16-13 Copyright © 2006, Oracle. All rights reserved.

Получение информации о кодировках

Используйте представление `NLS_DATABASE_PARAMETERS` для получения сведений о постоянных установках NLS базы данных, включая кодировку базы данных и национальную кодировку. Представление выводит как явно установленные значения, так и используемые в базе данных по умолчанию.

Задание языковых особенностей



Copyright © 2006, Oracle. All rights reserved.

Задание языковых особенностей

БД позволяет не только хранить и выбирать данные на большинстве современных языков, но и предоставляет дополнительные возможности. Так можно выдавать даты с использованием местных форматов времени и даты, а также сортировать текстовые данные в соответствии с местными представлениями. Коме того, база данных поддерживает такие локальные особенности, как представления чисел и денежных величин, которые определяются географическим месторасположением.

NLS-параметры определяют местные особенности клиента и сервера. Существует четыре способа задания параметров *национальной языковой поддержки* (*National Language Support – NLS*).

- Параметры инициализации на стороне сервера, задающие для сервера среду NLS по умолчанию (такая установка не действует на стороне клиента);
- Переменные окружения для клиента, задающие локальные особенности, заменяющие установки по умолчанию для сервера;
- Параметры команды `ALTER SESSION`, заменяющие установки по умолчанию для клиента или для сервера;
- Кодирование в функциях SQL языковых особенностей для приложения или запроса, заменяющее значения по умолчанию параметров NLS, установленных на сервере и клиенте, а также в команде `ALTER SESSION`.

Задание языковых особенностей для сеанса

- **Задание местных особенностей в переменной окружения NLS_LANG:**
  - **Язык**
  - **Территория**
  - **Кодировка**

```
NLS_LANG=RUSSIAN_CIS.CL8MSWIN1251
```

- **Задание других NLS-переменных окружения:**
  - **Переопределение параметров инициализации базы данных для всех сеансов**
  - **Настройка местных особенностей**
  - **Изменение местоположения по умолчанию для библиотечных файлов NLS**

ORACLE

16-15 Copyright © 2006, Oracle. All rights reserved.

Задание языковых особенностей для сеанса

Переменная окружения NLS_LANG

Местная специфика (*locale*) отражает лингвистическое и культурное окружение, в котором выполняется система или программа. Самый простой способ задания местных особенностей для программного обеспечения Oracle состоит в задании параметра окружения NLS_LANG. В нем определяется язык и территория, которые используются клиентским приложением и сервером базы данных, а также устанавливается кодировка, в которой вводятся и выводятся данные клиентской программой. Значение переменной NLS_LANG переопределяет значения параметров инициализации NLS.

Каждая составная часть переменной влияет на подмножество характеристик NLS:

```
NLS_LANG=<язык>_<территория>.<кодировка>
```

Язык используется для переопределения значения параметра NLS_LANGUAGE. Территория переопределяет значение параметра NLS_TERRITORY. Кодировка определяет схему кодирования символов, используемую клиентским приложением (обычно тем, с которым пользователь работает в окне терминала).

Все части переменной NLS_LANG необязательные; если нет их явного определения, используются значения по умолчанию. Перед заданием территории *необходимо* указать символ подчеркивания (_), а перед кодировкой – точку (.) Например:

```
NLS_LANG=_CIS
NLS_LANG=.CL8MSWIN1251
NLS_LANG=RUSSIAN.CL8MSWIN1251
```

Значения параметров, зависимые от языка и территории

Параметр	Значения по умолчанию
NLS_LANGUAGE	AMERICAN
NLS_DATE_LANGUAGE	AMERICAN
NLS_SORT	BINARY
NLS_TERRITORY	AMERICA
NLS_CURRENCY	$
NLS_DUAL_CURRENCY	$
NLS_ISO_CURRENCY	AMERICA
NLS_DATE_FORMAT	DD-MON-RR
NLS_NUMERIC_CHARACTERS	.,
NLS_TIMESTAMP_FORMAT	DD-MON-RRHH.MI.SSXFF AM
NLS_TIMESTAMP_TZ_FORMAT	DD-MON-RRHH.MI.SSXFF AM TZR

Copyright © 2006, Oracle. All rights reserved.

Значения параметров, зависимые от языка и территории

Параметры инициализации NLS_LANGUAGE и NLS_TERRITORY определяют значения по умолчанию других параметров, используемых базой данных Oracle. Можно явно переопределить значения параметров, зависящих от установленных значений в параметрах NLS_LANGUAGE и NLS_TERRITORY.

Параметр инициализации NLS_LANGUAGE

Параметр инициализации NLS_LANGUAGE задает значения по умолчанию следующих параметров:

Параметр	Описание
NLS_DATE_LANGUAGE	Явно изменяет язык для полных и сокращенных названий дней и месяцев и значения других элементов формата даты.
NLS_SORT	Изменяет последовательность лингвистической сортировки, используемую Oracle для сортировки символьных значений. (Значение должно соответствовать названию последовательности лингвистической сортировки).

Значения параметров, зависимые от языка и территории (продолжение)

Параметр инициализации NLS_TERRITORY

Параметр инициализации NLS_TERRITORY задает значения по умолчанию следующих параметров:

Параметр	Описание
NLS_CURRENCY	Задает символ местной денежной единицы
NLS_DATE_FORMAT	Задает формат даты (значение должно соответствовать модели формата даты)
NLS_DUAL_CURRENCY	Задает вторичный символ денежной единицы для данной территории
NLS_ISO_CURRENCY	Отражает территорию, символ денежной единицы которой в стандарте ISO должен использоваться
NLS_NUMERIC_CHARACTERS	Явно задает новые десятичный и групповой разделители
NLS_TIMESTAMP_FORMAT	Определяет формат даты по умолчанию для типов данных TIMESTAMP и TIMESTAMP WITH LOCAL TIME ZONE. **Должен быть установлен параметр NLS_LANG**
NLS_TIMESTAMP_TZ_FORMAT	Определяет формат даты по умолчанию для типа данных TIMESTAMP и TIMESTAMP WITH TIME ZONE, используемый с функциями TO_CHAR и TO_TIMESTAMP_TZ. **Должен быть установлен параметр NLS_LANG**

Следующие NLS-параметры инициализации зависят от параметров NLS_LANGUAGE и NLS_TERRITORY:

Параметр	Описание
NLS_CALENDAR	Задает календарь, используемый базой данных Oracle.
NLS_COMP	Может иметь значение ANSI или BINARY. Если NLS_COMP равен ANSI, операции SQL производят лингвистическое сравнение на основе значения параметра NLS_SORT.
NLS_LENGTH_SEMANTICS	Предоставляется возможность создания столбцов с типом CHAR, VARCHAR2 и LONG с указанием длины в байтах или символах.

Задание языковых особенностей

```
ALTER SESSION SET
NLS_DATE_FORMAT='DD.MM.YYYY';
```

```
DBMS_SESSION.SET_NLS('NLS_DATE_FORMAT',
'''DD.MM.YYYY''') ;
```

Использование NLS-параметров в функциях SQL

```
SELECT TO_CHAR(hire_date,'DD.Mon.YYYY',
 'NLS_DATE_LANGUAGE=RUSSIAN')
FROM employees
WHERE hire_date > to_date ('01-ЯНВ-2000',
'DD.Mon.YYYY','NLS_DATE_LANGUAGE=RUSSIAN')
```

ORACLE

16-18 Copyright © 2006, Oracle. All rights reserved.

Задание языковых особенностей

Отдельные характеристики NLS для сеанса изменяются с помощью команды `ALTER SESSION`. Для выполнения таких же действий можно использовать хранимую пакетную процедуру `DBMS_SESSION.SET_NLS`, передав в нее наименование и новое значение для изменяемого параметра. Во втором примере, приведенном на слайде, с помощью пакета `DBMS_SESSION` выполняется то же самое действие, что и в первом примере.

Клиентские утилиты (например, *i*SQL*Plus, SQL*Plus, SQL*Loader) читают переменные окружения на клиенте и выполняют соответствующие команды `ALTER SESSION` сразу после установления соединения с базой данных.

В третьем примере показано использование параметра `NLS_DATE_LANGUAGE` в функции SQL `TO_CHAR`.

Лингвистический поиск и сортировка

На порядок сортировки влияют :

- **Регистр символов**
- **Диакритические и находящиеся под ударением символы**
- **Комбинация символов, обрабатываемых как один символ**
- **Фонетика и символьное представление**
- **Культурные предпочтения**

ORACLE

16-19 Copyright © 2006, Oracle. All rights reserved.

Лингвистический поиск и сортировка

В различных языках свой порядок сортировки. Кроме того, в странах с различными культурными особенностями и одним и тем словарем слова могут сортироваться по-разному. Например, в датском языке, Æ следует за Z, в то время как Y и Ü рассматриваются как варианты одного и того же символа. Порядок сортировки:

- может зависеть и не зависеть от регистра символов;
- может игнорировать или учитывать диакритические знаки (отметка рядом или проходящая через символ или комбинацию символов, отражающая особенности произношения символа с и без диакритического знака);
- может зависеть от фонетики и основываться на внешнем представлении символа (например, число черточек в идеографическом письме Восточной Азии).

Другая общая проблема сортировки – объединение букв в единственный символ. Например, в традиционном испанском языке `ch` – отдельный символ, стоящий после `c`. В следующем примере слова упорядочены правильно: *cerveza*, *colorado*, *cheremoya*. Поэтому символ `c` нельзя правильно отсортировать, пока база данных не проверит, не стоит ли за ним буква `h`.

Для того, чтобы отсортировать символы по алфавиту, необходимо использовать метод, отличный от использования номера символа в таблице кодировки. Такой метод называют *лингвистической сортировкой* (*linguistic sort*). При его использовании символы заменяются номерами, которые отражают соответствующий лингвистический порядок этих символов.

Лингвистический поиск и сортировка

Три типа сортировки:

- **Двоичная сортировка**
  - **Сортировка выполняется в соответствии с двоичным значением кодов символов**
- **Одноязычная лингвистическая сортировка**
  - **Сортировка в два прохода, основанная на минорных и мажорных величинах, назначенных символам**
- **Многоязычная лингвистическая сортировка**
  - **Основывается на стандарте ISO (ISO 14651) и стандарте Unicode 3.2 для многоязычного сравнения**
  - **Сортировка с учетом черточек, PinYin (фонетическая транскрипция) и корней в китайских символах**

ORACLE

16-20 Copyright © 2006, Oracle. All rights reserved.

Лингвистический поиск и сортировка (продолжение)

Двоичная сортировка – это обычный механизм сортировки, в котором буквы сортируются на основе двоичных значений, используемых для кодирования символов. Двоичная сортировка – это самый быстрый тип сортировки. Для символов английского алфавита он дает верный результат, так как в стандартах ASCII и EBCDIC буквы от A до Z расположены в возрастающем порядке в таблице кодировки. Двоичная сортировка символов других языков обычно дает неверный результат.

При выполнении одноязычной сортировки Oracle использует мажорные и минорные величины для сравнения символов. Обычно буквы, внешне одинаковые, имеют одинаковые мажорные значения. Например, *E*, *e* и *ë*. Oracle определяет символы с диакритическими знаками и чувствительные к регистру, которые имеют одинаковые мажорные, но разные минорные значения.

Oracle выполняет одноязычную сортировку в два прохода, сравнивая сортируемые строки одного языка. В первом проходе сравниваются мажорные значения всех символов строк на основе мажорной таблицы, во втором проходе – минорные значения на основе минорной таблицы, что позволяет более правильно выполнить сортировку по сравнению с двоичной сортировкой. Однако такой подход все еще имеет ограничения.

Многоязычная сортировка позволяет сортировать информацию на нескольких языках в одной операции, что подходит для регионов или языков со сложными правилами сортировки.

Дополнительные сведения о поддерживаемых лингвистических сортировках см. в документе *Oracle Database Globalization Support Guide*.

Использование лингвистического поиска и сортировки

Тип сортировки символьных данных можно задать используя:

- **Параметр `NLS_SORT`**
  - **Значение параметра по умолчанию устанавливается на основе переменной окружения `NLS_LANG`, если она была определена**
  - **Параметр может быть задан в сеансе, на клиенте и сервере**
- **Функцию `NLSSORT`**
  - **Метод сортировки определяется в операторе запроса**

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Использование лингвистического поиска и сортировки

Для преодоления ограничений двоичной сортировки можно задать лингвистические сортировки с помощью параметра `NLS_SORT` или функции `NLSSORT` в запросе.

Параметр `NLS_SORT`

Рассмотрим следующие слова, хранимые в БД в кодировке `WE8ISO8859P1`:

- `gelée`
- `gelé`
- `gèle`
- `gelez`

Когда параметр `NLS_SORT` равен `BINARY`, в отсортированном наборе слово `gelez` стоит до `gelé`. Это происходит, потому что двоичный код символа `e` меньше кода символа `è` в схеме кодирования `WE8ISO8859P1`.

Если для параметра `NLS_SORT` установить значение `French`, тогда слова будут отсортированы в следующем порядке: `gelé`, `gelez`, `gèle`. `French` – это одноязычная сортировка, которая не охватывает все нюансы французского языка, например, сортировку французских букв слева направо и ударений справа налево.

При установке в параметре `NLS_SORT` значения `FRENCH_M` выполняется многоязычная сортировка, при которой правильно учитываются как символы, так и диакритические знаки.

Использование лингвистического поиска и сортировки (продолжение)

Функция NLSSORT

Функция NLSSORT предоставляет возможность задавать тип сортировки на уровне запроса. В следующем примере параметр NLS_SORT в сеансе равен BINARY, а затем изменяется в выполняемом запросе.

```
SQL> ALTER SESSION SET NLS_SORT=BINARY;
 Session altered.

 SQL> SELECT  fr_word
  2   FROM    words
  3   ORDER BY fr_word;

 FR_WORD
 -------
 gelez
 gelé
 gelée
 gèle

 SQL> SELECT  fr_word
  2   FROM    words
  3   ORDER BY NLSSORT(fr_word,'NLS_SORT=FRENCH_M');

 FR_WORD
 --------
 gèle
 gelé
 gelée
 gelez
```

В схеме кодирования CL8MSWIN1251 буква ё предшествует букве е. Поэтому результат двоичной сортировки слов *ель* и *ёлка* выглядит следующим образом:

```
SQL> ALTER SESSION SET NLS_SORT=BINARY;
 Session altered.

 SQL> SELECT  word
  2   FROM    rus_list
  3   ORDER BY word;

 WORD
 -----
 ёлка
 ель
```

После замены сортировки на лингвистическую результат меняется:

```
SQL> ALTER SESSION SET NLS_SORT=RUSSIAN;
 Session altered.

 SQL> SELECT  word
  2   FROM    rus_list
  3   ORDER BY word;

 WORD
 -----
 ель
 ёлка
```

Поиск и сортировка, не зависящие от регистра символов и ударения

- **Укажите наименование лингвистической сортировки:**

```
NLS_SORT = <NLS_имя_сортировки>[_AI | _CI]
```

- **Примеры:**

```
NLS_SORT = FRENCH_M_AI
```

```
NLS_SORT = XGERMAN_CI
```

- **Задайте для предложений WHERE и блоков PL/SQL операцию сравнения с учетом лингвистической сортировки:**

```
NLS_COMP = BINARY | ANSI
```

- **Такой поиск и сортировка могут быть полезны для баз данных, которые были мигрированы в Oracle**

ORACLE

16-23 Copyright © 2006, Oracle. All rights reserved.

Поиск и сортировка, не зависящие от регистра символов и ударения

Для задания названия лингвистической сортировки используется параметр NLS_SORT. По умолчанию его значение устанавливается на основе параметра NLS_LANGUAGE. Суффиксы AI и CI добавляются к наименованию сортировки, чтобы отобразить *сортировку, не зависящую от ударений* (*accent-insensitive sort* – AI) или *не зависящую от регистра символов* (*case-insensitive* – CI). CI задает сортировку, *не зависящую от регистра символов,* AI – *не зависящую от ударений и регистра символов.* В примере на слайде определены:

- не зависящая от ударений и не зависящая от регистра символов сортировка French_M;
- зависящая от ударений и не зависящая от регистра символов сортировка Xgerman.

Сравнения во фразе WHERE и блоках PL/SQL производятся в двоичном режиме, если только не используется функция NLSSORT. После установки в параметре NLS_COMP значения ANSI операции сравнения во фразе WHERE и блоках PL/SQL выполняются с использованием лингвистической сортировки, заданной в параметре NLS_SORT. Следует также определить индекс для столбца, на основе которого будут выполняться лингвистические сортировки.

Примечание. Формат параметра NLS_SORT применяется как для одноязычной, так и многоязычной сортировок. Одноязычная сортировка производится только с учетом мажорных и минорных значений символов. Когда в параметре NLS_COMP установлено значение ANSI, опции _AI и _CI применяются к любым операторам SQL и PL/SQL, которые поддерживали еще в предыдущих версиях базы данных сравнения, основанные на сопоставлении. Опции _AI и _CI не оказывают влияния на функции SQL INSTR, TRIM и LIKE, поскольку эти функции сравнивают строки, используя только двоичную сортировку.

Примечание для мигрированных баз данных. Sybase, SQL Server и MS Access могут использовать операции, не зависящие от регистра символов. Для сохранения существующих

функциональных возможностей приложения можно использовать опции `_AI` и `_CI`.

Поддержка поиска и сортировки в командах и функциях SQL

- **Следующие предложения SQL учитывают значения, установленные в параметрах NLS_SORT и NLS_COMP:**
  - **WHERE**
  - **ORDER BY**
  - **START WITH**
  - **HAVING**
  - **IN/NOT IN**
  - **BETWEEN**
  - **CASE-WHEN**
- **Функция NLSSORT() поддерживает поиск и сортировку, не зависящие от регистра символов и ударений.**

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Поддержка поиска и сортировки в командах и функциях SQL

Опции _AI и _CI, добавленные к параметру NLS_SORT, оказывают влияние на предложения SQL, перечисленные на слайде.

Функция NLSSORT обычно используется в предложении ORDER BY или WHERE команды SQL, когда в этой команде необходимо задать сортировку, отличную от установленной в параметре NLS_SORT. В приведенном ниже примере производится поиск всех “De Niro”, независимо от регистра символов и ударений.

```
SELECT cust_last_name  FROM customers
        WHERE  NLSSORT(cust_last_name, 'NLS_SORT = generic_m_ai‘)
        = NLSSORT('De Niro', 'NLS_SORT=generic_m_ai');
CUST_LAST_NAME
--------------------
de Niro
De Niro
dë Nirõ
```

Такой же результат можно получить, если следующим образом установить параметры NLS_COMP и NLS_SORT:

```
ALTER SESSION SET NLS_SORT=generic_m_ai;
ALTER SESSION SET NLS_COMP=ansi;
```

Пример с буквами е и ё для русского языка:

```
SELECT word FROM rus_list
WHERE NLSSORT(word, 'NLS_SORT = generic_m_ai')
        = NLSSORT('еж', 'NLS_SORT=generic_m_ai')
WORD
----------
еж
Еж
ёж
Ёж
```

Поддержка лингвистических индексов

- **Создайте индекс на основе лингвистически отсортированных значений**
- **Быстро выполняемый запрос данных без обязательного указания фразы `ORDER BY` или функции `NLSSORT`:**

```
CREATE INDEX list_word ON
 rus_list (NLSSORT(word, 'NLS_SORT=RUSSIAN'));

SELECT word FROM rus_list;
```

- **Задайте соответствующее значение параметра `NLS_SORT` перед созданием индекса для выполнения необходимой сортировки**

ORACLE

16-25 Copyright © 2006, Oracle. All rights reserved.

Поддержка лингвистических индексов

Лингвистическая сортировка зависит от языка. Если в БД хранится информация на разных языках, может потребоваться сортировка, учитывающая это. Создание лингвистического индекса для столбцов значительно повышает производительность запросов, требующих лингвистическую сортировку, но замедляет операции вставки и обновления.

Индексы с лингвистическими отсортированными данными создаются как индексы, основанные на функции. SQL-функция `NLSSORT` сортирует строки байтов, заданные в первом параметре, в соответствии с указанной лингвистической последовательностью. В примере на слайде создается индекс для значений столбца `WORD`, сортируемых по русскому алфавиту. После этого могут выполняться запросы, использующие индексы со значениями, отсортированными по правилам русского языка.

Можно также создать один лингвистический индекс для всех языков, используя одну из многоязыковых лингвистических сортировок, например, `GENERIC_M` или `FRENCH_M`. Или же для небольшого множества языков можно использовать столбец, в котором задается язык, передаваемый в качестве параметра функции `NLSSORT`. Столбец должен содержать значения, соответствующие правильным значениям параметра `NLS_LANGUAGE`, для индексируемого столбца.

```
CREATE INDEX i2 on list (NLSSORT(word, 'NLS_SORT=GENERIC_M'));
CREATE INDEX word_all_idx ON
         list(NLSSORT(word, 'NLS_SORT='||LANG_COL));
```

Дополнительные сведения о создании лингвистических индексов см. в документе *Oracle Database Globalization Support Guide*.

Настройка лингвистического поиска и сортировки

- Игнорируемые символы
- Сокращающиеся и расширяющиеся символы
- Специальная комбинация символов и специальные символы
- Расширяющиеся символы и специальные символы
- Специальные большие и маленькие буквы
- Контекстно-зависимые символы
- Обратная вторичная сортировка
- Каноническая эквивалентность

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Настройка лингвистического поиска и сортировки

Лингвистический поиск и сортировки имеют много возможностей. Большинство из них настраиваются для получения необходимого результата, например:

- Теперь можно задать сортировку или запрос на основе базовых символов (независимую от ударений) или же на основе базовых символов и ударений (независимую от регистра символов).
- Можно задать игнорирование символа тире, в результате чего значение `e-mail` обрабатывается как `email`.
- Расширяющийся символ `ö` рассматривается при сортировке как `oe` после `od` и перед `of`.
- Надлежащая сортировка знаков удлиненных звуков в японском языке.
- Замена `ä` на базовый эквивалент `a` и умляут (`¨`), в результате `ä` и `a¨` считаются одним и тем же.
- Расположение диакритического символа перед или после его непомеченного варианта.
- Можно задать, надо ли в тайском и лаосском языках сначала поменять местами некоторые символы и следующий за ними символ перед сортировкой.
- Отображение маленьких букв в несколько больших букв, например, в немецком `ß` в `SS`, а также отображение больших букв в несколько маленьких, например, в турецком `I` становится маленькой `i` без точки: `ı`.

Для настройки лингвистических сортировок необходимо использовать утилиту Oracle Locale Builder.

Неявные преобразования CLOB в NCLOB и NCLOB в CLOB

Неявное прозрачное преобразование поддерживается для:

- **связанных переменных SQL типа IN и OUT, задаваемых в запросах и командах DML**
- **передаваемых параметров функций и процедур PL/SQL**
- **операций PL/SQL, в которых переменной присваивается значение**

ORACLE

16-27 Copyright © 2006, Oracle. All rights reserved.

Неявные преобразования CLOB в NCLOB и NCLOB в CLOB

Все чаще требуется производить прямое преобразование данных, а также обратное из кодировки Unicode в национальную кодировку базы данных. Явное преобразование из одного типа данных в другой для CLOB и NCLOB обеспечивалось в SQL и PL/SQL с помощью функций TO_CLOB и TO_NCLOB. В базе данных Oracle 10*g* разрешается неявное преобразование для связанных переменных SQL типа IN и OUT, задаваемых в запросах и командах DML, а также для передаваемых параметров функций и процедур PL/SQL и

операций PL/SQL, в которых переменной присваивается значение. Например, в следующем примере выполняются полностью прозрачные преобразования между этими типами данных.

```
CREATE TABLE my_table (nclob_col NCLOB);
DECLARE
    clob_var CLOB;
    nclob_var NCLOB;
BEGIN
    clob_var := 'clob data'; -- инициализация CLOB
    -- Вставка значения CLOB в столбец NCLOB
      INSERT INTO my_table VALUES (clob_var);
    -- Определение переменной CLOB на основе столбца NCLOB
      SELECT nclob_col
            INTO clob_var FROM my_table;
END;
```

NLS-преобразования данных при использовании утилит Oracle

- **Несколько преобразований данных может быть выполнено при экспорте из одной БД и импорте в другую БД, когда не используется одна и та же кодировка.**
- **Внешние таблицы используют установки NLS на сервере для выявления кодировки данных.**
- **Загрузка с помощью SQL*Loader:**
  - **Обычная: данные преобразуются в кодировку сеанса, заданную параметром `NLS_LANG`.**
  - **Прямая: данные преобразуются с использованием директив на стороне клиента.**

ORACLE

16-28 Copyright © 2006, Oracle. All rights reserved.

NLS-преобразования данных при использовании утилит Oracle -

Поддержка глобализации для выполнения экспорта и импорта

Утилита Export всегда сохраняет данные, в том числе и данные в Unicode, в той же самой кодировке базы данных, из которой они были экспортированы. При экспорте описаний объектов утилита Export пишет в создаваемый файл команды на *языке определения данных* (*data definition language –DDL*) в кодировке, заданной в переменной окружения `NLS_LANG` для сеанса пользователя. Если кодировки в переменной `NLS_LANG` и БД различаются, выполняется преобразование.

Во время импорта, если кодировки исходной БД (и экспортного файла) отличаются от кодировок БД, в которую импортируются данные, выполняется автоматическое преобразование в кодировки целевой базы данных.

Когда импортируются DDL-описания, они автоматически преобразуются из кодировки экспортного файла в кодировку пользовательского сеанса, выполняющего импорт. Такое преобразование при импорте возможно только для однобайтных кодировок. Поэтому для многобайтных кодировок схема кодирования, используемая при импорте файла, должна совпадать с схемой кодирования экспортного файла. Если кодировка, используемая в пользовательском сеансе импорта отличается от кодировки целевой БД, выполняется окончательное преобразование.

Поддержка глобализации для внешних таблиц

Установки переменных среды NLS на **сервере** определяют кодировку и маски данных для таблицы.

NLS-преобразования данных при использовании утилит Oracle (продолжение)

Поддержка глобализации для утилиты SQL*Loader

SQL*Loader поддерживает четыре кодировки:

- кодировку на стороне клиента (NLS_LANG клиентского процесса SQL*Loader);
- кодировку файла данных (обычно совпадает с кодировкой на стороне клиента);
- кодировку базы данных;
- национальную кодировку базы данных.

Наивысшая производительность достигается, когда все кодировки одинаковые.

SQL*Loader имеет возможность преобразования данных из кодировки файла данных в кодировку базы данных. Кодировка файла данных задается переменной среды NLS_LANG или в параметре CHARACTERSET управляющего файла утилиты SQL*Loader:

```
LOAD DATA
CHARACTERSET UTF16
INFILE ulcase11.dat
REPLACE …
```

Предполагается, что сам управляющий файл SQL*Loader должен быть в кодировке, определяемой действующей в сеансе переменной NLS_LANG. Когда кодировка управляющего файла отличается от кодировки файла данных, ограничители и фразы сравнения, заданные в управляющем файле утилиты SQL*Loader в виде символьных строк, преобразуются из кодировки управляющего файла в кодировку файла данных перед выполнением любого сравнения. Чтобы быть уверенным в том, ограничители заданы верно, можно указать их шестнадцатеричные значения вместо символьных строк .

Когда кодировка, установленная в параметре NLS_LANG для вашего сеанса, отличается от кодировки файла данных, символьные строки в управляющем файле преобразуются в кодировку файла данных. Это выполняется до проверки утилитой SQL*Loader заданных стандартных терминаторов строк.

Кодировка, указанная в параметре CHARACTERSET, не применяется к данным в управляющем файле (кодировка задается вместе с фразой INFILE). Чтобы загрузить данные, представленные в кодировке, отличной от заданной в сеансе с помощью переменной NLS_LANG, необходимо поместить данные в отдельный файл данных.

Утилиту SQL*Loader используют для загрузки данных тремя способами: обычным (conventional path), прямым (direct path) или с помощью внешней таблицы (external table). В обычном режиме загрузки данные преобразуются в кодировку сеанса, заданную переменной NLS_LANG.

В прямом режиме загрузки преобразование данных происходит на стороне клиента, а не на сервере. Это означает, что параметры NLS из файла параметров инициализации не используются. Чтобы переопределить такое функционирование, можно задать форматную маску в управляющем файле SQL*Loader, что эквивалентно установке параметра NLS в файле параметров инициализации или соответствующей переменной окружения.

Когда целевая кодировка загружаемых утилитой SQL*Loader данных не является надмножеством кодировки исходного файла данных, символы, не имеющие эквивалента в целевой кодировке, преобразуются в символы замещения, например в вопросы, что приводит к потери данных.

NLS-преобразование данных при использовании утилиты Data Pump

- **Утилита Data Pump Export всегда сохраняет данные в кодировке базы данных, из которой они были извлечены.**
- **Утилита Data Pump Import преобразует данные в кодировку целевой базы данных, когда это необходимо.**
- **Журнальный файл утилиты Data Pump заполняется на языке, заданном в переменной `NLS_LANG`, действующей в сеансе, в котором была запущена утилита Data Pump.**

ORACLE

16-30 Copyright © 2006, Oracle. All rights reserved.

NLS-преобразование данных при использовании утилиты Data Pump

Параметры NLS сеанса, в котором была запущена утилита Data Pump, используются при выполнении задания этой утилиты. Поэтому перед запуском любого задания Data Pump убедитесь в том, что параметры установлены правильно. Переменная `NLS_LANG` влияет только на сообщения, возвращаемые утилитами Data Pump (`impdp` и `expdp`).

Во время выполнения задания дополнительно может заполняться журнальный файл протокола работы. В нем отражается продвижение задания, а также ошибки, если они происходят. Утилита Data Pump пишет в журнальный файл с учетом параметра `NLS_LANG` на стороне клиента. Например, установка `NLS_LANG` в `RUSSIAN.CL8MSWIN1251` приводит к записи всех сообщений задания (job) на русском языке, даже если задание было перезапущено с клиента, на котором установлено значение `AMERICAN.WE8DEC`

При использовании файла параметров (`PARFILE`) с утилитой Data Pump предполагается, что его содержимое в кодировке клиента . Утилита Data Pump транслирует текстовые строки в файлах параметров в кодировку базы данных. Когда в файле параметров информация содержится в кодировке, отличной от используемой в настоящее время на клиенте, необходимо изменить установки NLS на клиента для соответствующей трансляции.

Характеристики поддержки глобализации

- Поддержка национальных языков
- Поддержка территориальных особенностей
- Поддержка кодировок
- Лингвистические сортировки
- Поддержка сообщений на разных языках
- Форматы дат и времени
- Числовые форматы
- Форматы денежных единиц

ORACLE

16-31 Copyright © 2006, Oracle. All rights reserved.

Характеристики поддержки глобализации

Различия культурных традиций и географическое положение разных стран напрямую воздействуют на форматы данных. Поддержка глобализации (g*lobalization support*) обеспечивает автоматическую адаптацию к родному языку пользователя утилит базы данных, сообщений об ошибках, порядка сортировки, дат, времени, а также денежных, числовых и календарных соглашений. Пользователи могут сохранять, обрабатывать, извлекать данные, а также взаимодействовать друг с другом на родных языках.

- Временные зоны могут использоваться вместе с поддержкой летнего времени.
- Поддерживаются также национальные календари, такие как григорианский, японский, имперский и буддистский.
- Обозначения денежных единиц отражают локальную экономику и соглашения ISO. Обозначения кредита и дебета также отличаются в разных странах.

Поддержка глобализации в базе данных Oracle осуществляется с помощью *исполняемой библиотеки NLS* (*Oracle NLS Runtime Library – NLSRTL*). Эта библиотека предоставляет полный набор независимых от языка функций, обеспечивающих соответствующую обработку текстов и символов, а также преобразования на требуемом языке. Поведение этих функций, зависящее от языка и территории, управляется набором данных о местных особенностях, которые выявляются и загружаются при выполнении

Операции, зависимые от языка, управляются множеством параметров и переменных окружения, как на стороне клиента, так и на стороне сервера. Сервер и клиент могут работать в одном месте или на расстоянии. В случае, когда клиент и сервер используют различные кодировки, Oracle автоматически преобразует символьные данные из одной кодировки в другую.

Итоги

- Определение подходящей кодировки базы данных, удовлетворяющей вашим бизнес-требованиям
- Общие сведения о конфигурации поддержки глобализации
- Настройка языковых особенностей на уровне базы данных и для отдельных сеансов
- Задание в запросах различных лингвистических сортировок
- Выбор данных, соответствующих строке поиска, в которой игнорируется регистр символов и ударение

ORACLE

Copyright © 2006, Oracle. All rights reserved.

Обзор практического занятия: использование возможностей поддержки глобализации

- Проверка кодировки базы данных и национальной кодировки
- Выявление правильных значений NLS
- Установка параметров NLS

ORACLE

16-33 Copyright © 2006, Oracle. All rights reserved.

Обзор практического занятия

Примечание: при выполнении заданий этого практического занятия вы будете использовать *i*SQL*Plus.